However, we found that this was much less effective than KL divergence and even worse than using logit difference on the IOI task (Figure 5), when we used random ablations and recorded ROC curves.

Overall, we found that KL divergence was the least flawed of all metrics we tried in this project, and think more practitioners should use it given i) the problems with other metrics listed, and ii) how empirically it can recover several circuits that were found by researchers using other metrics.

D Details of Subnetwork Probing and Head Importance Score for Pruning

D.1 Subnetwork Probing

There are 3 modifications we made to Subnetwork Probing (Cao, Sanh, and Rush, 2021, SP) in our work. In this Appendix we provide techincal detail and motivation for these modifications:

1. **We do not train a probe**. ACDC does not use a probe. Cao, Sanh, and Rush (2021) train a linear probe after learning a mask for every component. The component mask can be optimized without the probe, so we just omit the linear probing step.
2. **We change the objective of the SP process to match ACDC's**. ACDC uses a task-specific metric, or the KL divergence to the model's outputs (Algorithm 1). In order to compare the techniques in equivalent settings we use the same metric (be it KL divergence or task-specific) in SP. Cao, Sanh, and Rush (2021) use negative log probability loss.
3. **We generalize the masking technique so we can replace activations with both zero activations and corrupted activations**. Replacing activations with zero activations[6] is useful for pruning (as they improve the efficiency of networks) but are not as commonly used in mechanisitic interpretability (Goldowsky-Dill et al., 2023), so we adapt SP to use corrupted activations. SP learns a mask Z and then sets the weights ϕ of the neural network equal to $\phi * Z$ (elementwise multiplication), and locks the attention mask to be binary at the end of optimization (Jang, Gu, and Poole, 2017). This means that outputs from attention heads and MLPs in models are scaled closer to 0 as mask weights are decreased. To allow comparison with ACDC, we can linearly interpolate between a clean activation when the mask weight is 1 and a corrupted activation (i.e a component's output on the datapoint x_i', in the notation of Section 3) when the mask weight is 0. We do this by editing activations rather than weights of the model.

Additionally, we used a constant learning rate rather than the learning rate scheduling used in Cao, Sanh, and Rush (2021).

The regularization coefficients λ (in the notation of Cao, Sanh, and Rush (2021)) we used in Figure 4 were 0.01, 0.0158, 0.0251, 0.0398, 0.0631, 0.1, 0.158, 0.251, 0.398, 0.631, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 30, 50, 70, 90, 110, 130, 150, 170, 190, 210, 230, 250.

The number of edges for subgraphs found with Subnetwork Probing are computed by counting the number of edges between pairs of unmasked nodes.

D.2 Head Importance Score for Pruning

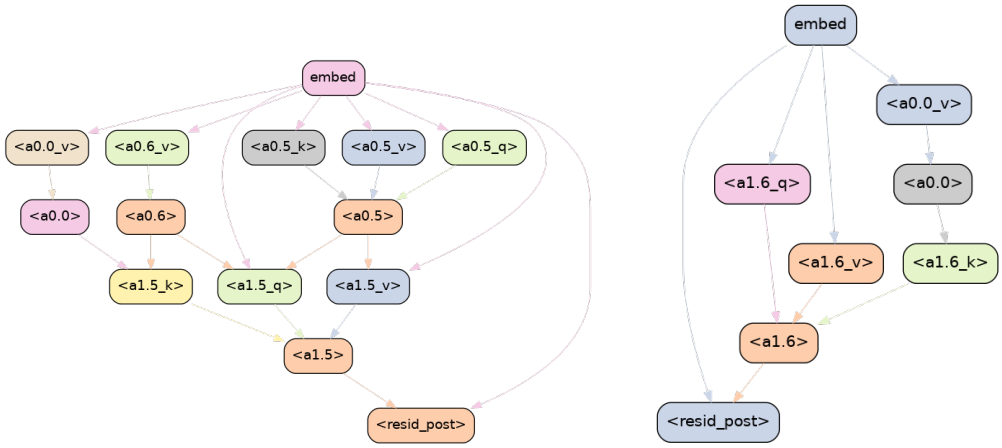
In Section 4-4.2 we compared ACDC with the Head Importance Score for Pruning (Michel, Levy, and Neubig, 2019, HISP). We borrow the author's notation throughout this section, (particularly from Section 4.1) and adapt it so that this Appendix can be read after Section 2.

The authors use masking parameters ξ_h for all heads, i.e scaling the output of each attention head by ξ_h, similar to the approach in Subnetwork Probing (Appendix D.1), so that each head Att_h's output is $\xi_h \text{Att}_h(x_i)$ on an input x_i. The authors keep MLPs present in all networks that they prune. We generalize their method so that it can be used for any component inside a network for which we can take gradients with respect to its output.

The authors define head importance as

$$I_h := \frac{1}{n} \sum_{i=1}^{n} \left| \frac{\partial \mathcal{L}(x_i)}{\partial \xi_h} \right| = \frac{1}{n} \sum_{i=1}^{n} \left| \text{Att}_h(x_i)^T \frac{\partial \mathcal{L}(x_i)}{\partial \text{Att}_h(x_i)} \right|. \tag{3}$$

[6]Which is generally equivalent to setting weight parameters equal to 0.



(a) Corrupted activations

(b) Zero activations

Figure 6: Examples of subgraphs recovered by ACDC on the induction task with different types of activations and threshold $\tau = 0.5623$. These find the two different induction heads (1.5, 1.6) and a previous token head (0.0) as identified by Goldowsky-Dill et al. (2023). (a) shows the result with corrupted activations, while (b) shows the result with zero activations.

where the equivalence of expressions is the result of the chain rule. We make three changes to this setup to allow more direct comparison to ACDC: i) we use a metric rather than loss, ii) we consider corrupted activations rather than just zero activations and iii) we use the 'head importance' metric for more internal components than merely attention head outputs.

Since our work uses in general uses a metric F rather than loss Appendix C, we instead use the derivative of F rather than the derivative of the loss. The HISP authors only consider interpolation between clean and zero activations, so in order to compare with corrupted activations, we can generalize ξ_h to be the interpolation factor between the clean head output $\text{Att}_h(x)$ (when $\xi_h = 1$) and the corrupted head output $\text{Att}_h(x')$ (when $\xi_h = 0$). Finally, this same approach works for any internal differentiable component of the neural network.[7] Therefore we study the HISP applied to the query, key and value vectors of the model and the MLPs outputs.

In practice, this means that we compute component importance scores

$$I_C := \frac{1}{n} \sum_{i=1}^{n} \left| (C(x_i) - C(x_i'))^T \frac{\partial F(x_i)}{\partial C(x_i)} \right|. \tag{4}$$

Where $C(x_i)$ is the output of an internal component C of the transformer, which is equivalent to 'attribution patching' (Nanda, 2023) up to the absolute value sign.

To compute the importance for zero activations, we adjust Equation (4) so it just has a $C(x_i)$ term, without the $-C_h(x_i')$ term. We also normalize all scores for different layers as in Michel, Levy, and Neubig (2019). The identical setup to Equation (4) works for outputs of the query, key and value calculations for a given head, as well as the MLP output of a layer. In Section 4 we use query, key and value components for each head within the network, as well as the output of all MLPs.

The number of edges for subgraphs found with HISP is also computed by counting the number of edges between pairs of unmasked nodes, like Subnetwork Probing (Appendix D.1).

E Experimental study of algorithm design

This section evaluates design choices for ACDC and SP, by re-doing the experiments in Section 4. We explore two axes of variation.

[7] In theory. In practice, components need be `torch.nn.Modules` such that we can calculate the gradient of F with respect to the components' outputs.

Table 2: AUCs for corrupted activations, Random Ablation. (E)=Edge, (N)=Node.

Metric	Task	ACDC(E)	HISP(E)	SP(E)	ACDC(N)	HISP(N)	SP(N)
KL	Docstring	**0.982**	0.805	0.937	**0.950**	0.881	0.928
	Greaterthan	**0.853**	0.693	0.806	**0.890**	0.642	0.827
	IOI	**0.869**	0.789	0.823	**0.880**	0.668	0.842
Loss	Docstring	**0.972**	0.821	0.942	0.938	0.889	**0.941**
	Greaterthan	0.461	0.706	**0.812**	0.766	0.631	**0.811**
	IOI	0.589	**0.836**	0.707	0.777	0.728	**0.797**
	Tracr-Proportion	**0.679**	**0.679**	0.525	0.750	**0.909**	0.818
	Tracr-Reverse	0.200	**0.577**	0.193	0.312	**0.750**	0.375

Table 3: AUCs for corrupted activations, Zero Ablation. (E)=Edge, (N)=Node.

Metric	Task	ACDC(E)	HISP(E)	SP(E)	ACDC(N)	HISP(N)	SP(N)
KL	Docstring	**0.906**	0.805	0.428	0.837	**0.881**	0.420
	Greaterthan	**0.701**	0.693	0.163	**0.887**	0.642	0.134
	IOI	0.539	**0.792**	0.486	0.458	**0.671**	0.605
Loss	Docstring	**0.929**	0.821	0.482	0.825	**0.889**	0.398
	Greaterthan	0.491	**0.706**	0.639	**0.783**	0.631	0.522
	IOI	0.447	**0.836**	0.393	0.424	**0.728**	0.479
	Tracr-Proportion	**1.000**	0.679	0.829	**1.000**	0.909	**1.000**
	Tracr-Reverse	**1.000**	0.577	0.801	**1.000**	0.750	**1.000**

- Minimizing the task-specific metric, rather than the KL divergence.
- Patching activations with zeros, rather than with the result on a corrupted input (interchange intervention).
- Looking at node-level TPR and FPR for the ROC curves, rather than edge-level.

The results paint a mixed picture of whether ACDC or SP is better overall, but reinforce the choices we implicitly made in the main text. A stand-out result is that ACDC with zero-patching is able to perfectly detect the `tracr` circuits (Figs. 9 and 10).

A numerical summary of the results is in Tables 2 and 3, which display the areas under the ROC curve (AUC) for all the design choices we consider.

E.1 Minimizing the task-specific metric, rather than the KL divergence

We ran ACDC, SP and HISP with the task-specific metric from Table 1, instead of KL divergence. The exact modification is described in Appendix C. The ROC result is in Fig. 7. Compared to minimizing KL divergence (Fig. 3), ACDC works better for Docstring, but worse for Greater-Than and IOI, indicating that it is not a very robust method.

We prefer using the KL divergence instead of the task-specific metric, because the task-specific metric can be over-optimized (Appendix C). This means that the recovered circuit ends up performing the task more than the original model, and is thus not accurate. We can observe this effect by comparing the task-specific metric achieved by the methods in Fig. 8.

E.2 Activation patching with zeros, instead of corrupted input

In the main text experiments that compared using corrupted activations and zero activations (Figure 4), all three methods recovered subgraphs with generally lower loss when doing activation patching with zeros, in both the experiments with the normal model and with permuted weights. It is unclear why the methods achieve better results with corruptions that are likely to be more destructive. A possible explanation is that there are 'negative' components in models (Appendix F.3) that are detrimental to the tasks, and the zero activations are more disruptive to these components. A discussion of how methods could be adjusted to deal with this difficulty can be found in Alternative 2 in Appendix C.